INSTITUTO ISIL

DESARROLLO DE APLICACIONES EMPRESARIALES AVANZADO

PROCESO DE APRENDIZAJE 2 (PA2)

Integración de devolución de stock y anulación trazable de pedidos

Estudiante: [APELLIDOS Y NOMBRES]
Docente: [NOMBRE DEL DOCENTE]
NRC: [NRC]
Periodo: 2026-20

BORRADOR: completar datos personales y resultados reales de pruebas antes de entregar.

1. Objetivo

Ampliar la integración entre inventario-service (Jakarta EE) y sistema-pedidos-springboot (Spring Boot) mediante una API REST de devolución de stock, un cliente HTTP desacoplado y capas Controller, Service y Repository para registrar anulaciones.

2. Cambios implementados

Inventario — Jakarta EE

Se incorporaron los DTO DevolucionStockRequest y DevolucionStockResponse y el endpoint POST /api/inventario/productos/codigo/{codigo}/devoluciones. ProductoResource delega en InventarioService.devolverStockPorCodigo, que valida cantidad positiva, busca el producto y modifica stock dentro de la transacción local EJB. ProductoInventario conserva el campo @Version para control optimista.

Pedidos — Spring Boot

Se añadieron DevolucionStockClient y DevolucionStockRestClient con RestClient, DTO de transporte, URL base configurable y manejo de errores HTTP/conexión. Pedido incorpora PedidoEstado (REGISTRADO/ANULADO). AnulacionPedido registra pedidoId, código de producto, cantidad devuelta, fecha y motivo; la restricción UNIQUE por pedidoId ayuda a evitar anulaciones repetidas. El nuevo Controller expone POST y GET /api/v1/anulaciones.

3. Arquitectura

```
Cliente HTTP / Postman
       |
       v
AnulacionPedidoController
       |
       v
AnulacionPedidoServiceImpl ---- PedidoRepository ---- H2
       |                         AnulacionPedidoRepository -- H2
       v
DevolucionStockClient -> DevolucionStockRestClient
       | HTTP / JSON
       v
ProductoResource -> InventarioService (@Stateless)
       -> ProductoInventario (JPA) -> SQL Server
```

La configuración completa del diagrama editable está en documentacion/diagrama-arquitectura-pa2.md.

4. Inyección de dependencias y transacciones

En Inventario, @EJB permite que WildFly inyecte el EJB InventarioService administrado por el contenedor. En Spring, los servicios y controladores reciben dependencias por constructor; el contenedor IoC resuelve repositorios y clientes. Esta técnica reduce el acoplamiento y permite probar la lógica con mocks.

La transacción EJB protege el cambio de stock en SQL Server, mientras que @Transactional administra las escrituras locales en H2. Son transacciones separadas: @Transactional no revierte una operación HTTP ya confirmada en Inventario. Si la devolución remota falla, el flujo no registra la anulación local. Si Inventario confirma y luego falla una escritura local, el sistema necesita conciliación o compensación. No se configuran reintentos automáticos del POST.

5. Pruebas unitarias agregadas

Inventario: (1) devolución válida incrementa stock, por ejemplo 8 + 2 = 10; (2) cantidad cero o negativa se rechaza y el stock permanece igual. Pedidos: (1) anulación correcta llama una sola vez al cliente remoto, cambia el pedido a ANULADO y guarda auditoría; (2) anulación duplicada se rechaza sin devolución adicional; (3) excepción del cliente remoto no guarda anulación ni cambia el estado local.

Resultados de ejecución: [Completar luego de ejecutar mvn test en ambos proyectos. No afirmar éxito sin comprobar la salida real.]

6. Pruebas funcionales y evidencias pendientes

Adjuntar capturas reales de: anulación correcta con stock antes/después; anulación duplicada (409); pedido inexistente (404); Inventario detenido (503 y pedido aún REGISTRADO); devolución directa con cantidad válida e inválida (200/400). Incluir los resultados de mvn test en ambos proyectos y verificar el estado en H2.

7. Limitaciones y conclusión

El flujo secuencial evita la duplicación mediante la validación del estado y una restricción UNIQUE por pedidoId. Esta solución no garantiza idempotencia distribuida bajo solicitudes concurrentes ni atomicidad entre H2 y SQL Server. El reto opcional de compensación no está demostrado en este paquete y no debe declararse como completado sin una ejecución reproducible.

La implementación separa la API remota de la lógica de negocio y de la persistencia local, aplica Constructor Injection en Spring y conserva el límite transaccional de cada aplicación. Antes de entregar, se deben ejecutar las pruebas, completar evidencias, confirmar el funcionamiento real de los endpoints y actualizar este informe con resultados observados.

Referencias (APA)

Eclipse Foundation. (s. f.). *Jakarta RESTful Web Services*. https://jakarta.ee/specifications/restful-ws/

Eclipse Foundation. (s. f.). *Jakarta Enterprise Beans*. https://jakarta.ee/specifications/enterprise-beans/

VMware. (s. f.). *Spring Framework Reference Documentation: REST Clients*. https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

VMware. (s. f.). *Spring Data JPA*. https://docs.spring.io/spring-data/jpa/reference/

Lista final: completar carátula; ejecutar pruebas; adjuntar capturas; subir los dos proyectos a un repositorio Git personal accesible; reemplazar este borrador si el docente exige formato distinto.